wO={0.2, 0.2}, bF={0.0, 0.0}, bI={0.0, 0.0}, bC={0.0, 0.0}, bO={0.0, 0.0} und U=0.1 und input={1,0.5}. Verwende bei den Gates auch die Aktivierungsfunktionen

10:14

Hier ist die schrittweise Berechnung für eine LSTM-Zelle basierend auf deinen Werten. Für die Berechnung nehmen wir an, dass der vorherige **Hidden State** ($h_{t-1}$) und der **Cell State** ($c_{t-1}$) zu Beginn $0$ sind.

## Gegebene Parameter

- **Input ($x_t$):** $[1, 0.5]$
- **Gewichte ($W$):** $W_f = \{0.5, 0.5\}, W_i = \{0.1, 0.1\}, W_c = \{0.1, 0.1\}, W_o = \{0.2, 0.2\}$
- **Rekurrente Gewichte ($U$):** $0.1$ (für alle Gates angenommen)
- **Biases ($b$):** Alle $0.0$

---

## 1. Berechnung des Forget Gates ($f_t$)

Das Forget Gate entscheidet, was aus dem alten Cell State gelöscht wird.

$$f_t = \sigma(W_f \cdot x_t + U \cdot h_{t-1} + b_f)$$

$$f_t = \sigma((0.5 \cdot 1 + 0.5 \cdot 0.5) + 0.1 \cdot 0 + 0) = \sigma(0.75)$$

$$\sigma(0.75) = \frac{1}{1 + e^{-0.75}} \approx \mathbf{0.6792}$$

## 2. Berechnung des Input Gates ($i_t$) und Cell Candidates ($\tilde{c}_t$)

Das Input Gate bestimmt, welche neuen Informationen gespeichert werden.

$$i_t = \sigma(W_i \cdot x_t + U \cdot h_{t-1} + b_i) = \sigma(0.1 \cdot 1 + 0.1 \cdot 0.5) = \sigma(0.15) \approx \mathbf{0.5374}$$

Der Cell Candidate ist der potenzielle neue Inhalt:

$$\tilde{c}_t = \tanh(W_c \cdot x_t + U \cdot h_{t-1} + b_c) = \tanh(0.15) \approx \mathbf{0.1489}$$

## 3. Update des Cell State ($c_t$)

Kombination aus altem Wissen und neuen Informationen.

$$c_t = f_t \cdot c_{t-1} + i_t \cdot \tilde{c}_t$$

$$c_t = 0.6792 \cdot 0 + 0.5374 \cdot 0.1489 \approx \mathbf{0.0800}$$

## 4. Berechnung des Output Gates ($o_t$)

---

## ✅ Ergebnisübersicht

Der berechnete Zustand der LSTM-Einheit für den gegebenen Input ist:

- **Forget Gate ( ):**
- **Input Gate ( ):**
- **Cell State ( ):**
- **Output Gate ( ):**
- **Hidden State ( ):**

Möchtest du sehen, wie sich die Werte verändern, wenn wir einen **zweiten Zeitschritt** mit diesem Hidden State berechnen?

Show Code

KI-Antworten können Fehler enthalten. Weitere Informationen

1 Website

**Part II: Building LSTM from Scratch | by Samina Amin - Medium**

Long Short Term Memory (LSTM) — Step-by-Step Explanation and Python Implementation. Samina Amin. Follow. 5 min read. ·. Oct 13, 20...



Medium

Berechne die Backpropagation

Um die Backpropagation zu berechnen, bestimmen wir die Gradienten der Verlustfunktion bezüglich der Gewichte. Wir nehmen für dieses Beispiel an, dass der Fehler am Ausgang beträgt.

### 1. Gradient des Output Gates ( )

Zuerst berechnen wir, wie stark der Output-Zustand das Ergebnis beeinflusst:

Der Gradient vor der Aktivierungsfunktion (Sigmoid) ist:

- **Input Gate:**

- **Cell Candidate:**

- **Forget Gate:** Da ist, hat das Forget Gate in diesem ersten Schritt keinen Einfluss auf das Ergebnis ( ).

## 4. Update der Gewichte ( )

Die Gradienten für die Gewichte berechnen sich aus :

| Gewicht | Berechnung ( ) | Resultat |
|---|---|---|
| **Input Gate** | | |
| **Cell Candidate** | | |
| **Output Gate** | | |
| **Forget Gate** | | |

✅ **Zusammenfassung**
Die stärkste Gewichtsanpassung erfolgt beim **Cell Candidate ( )**, da dieser den größten direkten Einfluss auf den neuen Cell State und damit auf das Endergebnis hat.

Möchtest du nun wissen, wie diese Gradienten genutzt werden, um die **Gewichte mit einer Learning Rate** zu aktualisieren?

Show Code

KI-Antworten können Fehler enthalten. Weitere Informationen

3. Gradienten der internen Komponenten, Forget Gate, wie sieht das Ergebnis aus bei c(t-1)= 1

Wenn wir davon ausgehen, dass der vorherige Cell State (statt 0) war, ändert sich die Fehlerfortpflanzung für das **Forget Gate** entscheidend, da es nun aktiv am Ergebnis beteiligt ist.

Hier ist die Berechnung für :

## 2. Gradient vor der Aktivierungsfunktion ( )

Um den Gradienten vor der Sigmoid-Aktivierung zu erhalten, multiplizieren wir mit der Ableitung der Sigmoid-Funktion . Aus der Vorwärtsrechnung wissen wir:

.

Nun berechnen wir :

## 3. Update der Gewichte für das Forget Gate ( )

Die Anpassung der Gewichte berechnet sich wieder durch die Multiplikation mit dem Input :

**Ergebnis:**
Bei beträgt der Gradient für das Forget-Gate-Netzwerk **0.1244**. Das bedeutet, das Modell würde die Gewichte nun deutlich anpassen, um zu lernen, wie viel Information aus dem vorherigen Schritt für die Zukunft behalten werden soll.

Soll ich dir zeigen, wie sich durch den Wert auch der **neue Cell State** in der Vorwärtsrechnung verändert hätte?

KI-Antworten können Fehler enthalten. Weitere Informationen